

From Prompt to Experiment: Evaluating the Reliability of AutoRecLab 0.1 in Recommender Systems Research

Nicolas Lopez
nicolas.lopez@student.uni-siegen.de
University of Siegen
Siegen, Germany

Brael Namekong Signing
brael.namekongsigning@student.uni-siegen.de
University of Siegen
Siegen, Germany

Amine Bouhnini
amine.bouhnini@student.uni-siegen.de
University of Siegen
Siegen, Germany

ABSTRACT

AutoRecLab is a recently proposed LLM-driven autonomous research agent designed to automate recommender-systems experiments. In this work, we systematically evaluate its execution success, methodological correctness, and LLM backend performance using the README-suggested seed-sensitivity prompt, which is designed to conduct a seed-sensitivity analysis on LensKit’s ImplicitMF using MovieLens 100K. Across 13 initial end-to-end runs conducted across two execution environments and four OpenAI GPT-5-family backends, AutoRecLab achieved an internal satisfaction score (score ≥ 80 or accepted node) in 8 runs (61.5%). However, rigorous manual verification revealed that only 5 runs (38.5%) achieved true Methodological Success.

Across these methodologically successful runs, NDCG@10 ranged between 0.1910 and 0.2353, and Recall@10 ranged between 0.1703 and 0.3461, with seed variability yielding low coefficients of variation (0.8% to 2.5%). All methodologically successful runs held data splits fixed and evaluated model and training seed stochasticity, reproducing the core finding that random seeds impact recommender system performance while revealing autonomous protocol drift. The 8 remaining initial runs failed due to execution failures (API endpoint errors and platform process crashes) or methodological failures (unprompted custom SGD algorithm fallbacks and silent metric failures, such as applying `sklearn.metrics.ndcg_score` to raw score vectors). Descriptively, stronger models like gpt-5.4 consistently achieved higher execution success rates than smaller backends.

Our findings demonstrate that while AutoRecLab at this stage is a powerful exploratory tool, execution success and high agent reviewer scores do not guarantee Methodological Success, making human auditing essential and indispensable.

KEYWORDS

Autonomous research agents, Recommender systems, Reproducibility, Large language models

1 INTRODUCTION

1.1 Background

Recommender systems are central to many modern information services, and the corresponding research community has produced a rich body of algorithms, datasets, and evaluation protocols. In parallel, recent advances in large language models (LLMs) have led to the emergence of so-called *autonomous research agents* that aim to automate substantial parts of the scientific process. Examples include the AI Scientist by Sakana AI [5], the AI Co-Scientist by

Google DeepMind [3], Agent Laboratory by Schmidgall et al. [6], and CodeScientist [4]. While these systems target science in general, AutoRecLab [2] brings the same ambition to recommender systems: it translates a free-form research idea into an experiment plan, generates and executes Python code via tree search, and evaluates the result against the original requirements.

1.2 Research Problem

Manually conducting recommender-systems experiments is labor-intensive and error-prone, leaving many published findings difficult or impossible to reproduce. Existing automation tools such as AutoSurprise, LensKit-Auto, and LibRec-Auto are limited to hyperparameter tuning and do not cover experiment design, code generation, or reporting. AutoRecLab promises to close this gap, but was published as an exploratory proof of concept. It is therefore an open question whether an external team can run AutoRecLab end-to-end, reproduce its reported execution success rate, and verify whether its generated experiments achieve true methodological success. High execution success rates and high internal agent reviewer scores can mask underlying scientific errors, such as silent metric failures or unprompted protocol drift. Without rigorous independent auditing, claims regarding the reproducibility and reliability of autonomous research agents remain weakly supported.

1.3 Original Work

The original work by Beel, Gipp et al. [2] introduces AutoRecLab, an end-to-end autonomous research lab for recommender systems. The system is organised around four cooperating components. A *Planner* converts the user’s natural-language prompt into an explicit experiment specification (task, dataset, split protocol, baselines, metrics) and a binary requirements list (e.g., reproducible split, leakage check, seed control). A *Coder* generates Python code that fulfils these requirements, executes it in a sandbox, and is automatically re-invoked with stack traces when execution fails. A *Reviewer* numerically scores each candidate node against the requirements; a node that crashes scores zero. A *Tree-Search* module manages exploration across iterations: it picks the best node so far and, with probability ϵ , branches into less-promising nodes to escape local optima.

To demonstrate feasibility, the authors run a case study on the effect of random data-splitting seeds, using three algorithms (ALS, ItemKNN, Most Popular) from LensKit on three datasets (MovieLens 100K, Last.FM, Amazon Video Games) with five seeds per configuration. Four end-to-end runs are reported, of which three succeed (75%) at a marginal LLM cost of \$1–\$5 per run and laptop-scale runtimes of 2–9.5 h.

At the sub-experiment level, 15 of 27 individual configurations produced valid results (56%). Notably, all nine ItemKNN sub-experiments failed in the same way: AutoRecLab misconfigured the algorithm for implicit feedback without raising a runtime error, so the tree-search did not detect the problem. ALS and Most Popular, by contrast, closely tracked the human baseline. The authors explicitly position AutoRecLab as a proof of concept, arguing that the qualitative findings replicate human-level conclusions, but that silent misconfigurations remain a key open problem.

More importantly, the study demonstrates that autonomous recommender-systems research is already feasible. AutoRecLab was able to independently plan, implement, execute, and analyse recommender-systems experiments while arriving at scientifically meaningful conclusions. Based on these results, the authors argue that the recommender-systems community should pay greater attention to autonomous research agents and explore their potential to accelerate future scientific discovery and improve research reproducibility.

1.4 Research Goal

This work evaluates AutoRecLab’s ability to autonomously design, execute, and analyze a seed-sensitivity experiment on MovieLens 100K using LensKit’s ImplicitMF. We evaluate the agent across two primary dimensions: Execution Success (whether generated scripts execute without runtime crashes across operating systems and LLM backends) and Methodological Success (whether generated code correctly implements algorithms, ranking metrics, and evaluation protocols without silent errors or protocol drift).

Crucially, our analysis explicitly distinguishes between data-split sensitivity (varying the random seed used to split data into train and test sets across iterations) and model/training seed stochasticity (varying optimization seeds on a fixed data split).

To guide our evaluation, we formulate three core research questions:

RQ1 (Execution Success): How reliably does AutoRecLab achieve technical execution success across different LLM backends and execution environments, and what runtime failure modes occur?

RQ2 (Methodological Success): To what extent do autonomous runs achieve methodological success, and can AutoRecLab’s internal reviewer score reliably detect silent metric or algorithmic errors?

RQ3 (Seed Sensitivity Reproduction): Does AutoRecLab successfully reproduce the finding that random seeds impact evaluation metrics, and which source of randomness do autonomous runs evaluate?

This work is done as part of the Machine Learning Lab [1].

2 METHODOLOGY

2.1 Experiment Design and Execution Environment

To evaluate AutoRecLab, we conducted 13 initial end-to-end runs using the example prompt provided in the AutoRecLab README.md. The prompt instructs AutoRecLab to perform a seed-sensitivity analysis on LensKit’s ImplicitMF algorithm using the MovieLens 100K u. data dataset. The exact prompt, used verbatim by every team member, was:

I’d like to run an experiment to quantify how much random seeds affect recommender system results. Please use a simple ImplicitMF algorithm from LensKit, perform a train/test split on the MovieLens100K u. data, train it multiple times with identical settings but different random seeds, and analyze how metrics like NDCG or Recall vary across runs to find out if reporting seeds is important.

We deliberately did not modify this prompt across team members so that any observed variability does not come from prompt engineering. All 13 runs reported in this work were performed against the main branch of AutoRecLab, representing the early proof-of-concept version used in our experiments.

Execution Environments & Operating Systems.

We executed AutoRecLab across two distinct environment configurations to observe system behavior and platform compatibility:

- **Linux Containerized Environment (Docker):** Isolated Debian/Linux Docker container running Python 3.11.8 with pre-installed LensKit dependencies (lenskit, binpickle, scikit-learn, pandas, numpy). Subprocesses were executed within native Linux process forks.
- **Native Windows 11 Environment:** Local host execution on Windows 11 Pro (Python 3.11.5) using native Windows process spawning (spawn context) for executing agent-generated scripts and handling sub-process isolation.

LLM Backend Model Suite.

We systematically evaluated four OpenAI GPT-5-family models as the backend engine for AutoRecLab’s search planner, code generator, and reviewer module:

- **gpt-5.4 (3 runs):** OpenAI’s full-capability flagship model for complex code synthesis and reasoning.
- **gpt-5.4-mini (4 runs):** A lightweight, higher-throughput variant of gpt-5.4.
- **gpt-5-nano (3 runs):** A low-latency, highly compressed model variant.
- **gpt-5.3-codex (3 runs):** A specialized code-generation backend accessed via OpenAI chat completion endpoints.

Experimental Rules & Human Non-Intervention.

To strictly evaluate the autonomous capabilities of AutoRecLab, we enforced a strict zero-human-intervention policy during the execution loop. Once a prompt was submitted, our team performed no manual code edits, environment variable injections, or file fixes while the agent search tree was active. Human auditing and code inspection occurred exclusively after a run had terminated, either via success, crash, or reaching maximum tree search depth.

Experimental Unit Definitions.

To maintain clarity across our evaluation, we define five core experimental units:

- **End-to-End Run:** A single complete invocation of AutoRecLab, from initial prompt submission until the agent terminates naturally, crashes, or exhausts its node budget.
- **Node (Candidate Implementation):** A distinct Python code script (runfile.py) generated and tested by AutoRecLab during one iteration of its search tree.

- **Agent Reviewer Score:** The scalar score (0–100) assigned to a node by AutoRecLab’s internal LLM reviewer. Nodes scoring ≥ 80 are marked as satisfactory by the agent.
- **Manual Rerun Verification:** Re-executing a generated `runfile.py` script independently outside the AutoRecLab loop to verify execution stability and numerical exactness.
- **Seed Result:** The specific evaluation metrics, e.g., NDCG@10 and Recall@10 , produced by running a generated script for a single random seed.

Agent Configuration & Tree-Search Parameters.

All runs were executed using the default configuration file (`config.toml`) of the Main branch, defined by the following tree-search parameters:

- **num_draft_nodes = 3:** The agent generates three initial candidate implementations in parallel during the drafting phase.
- **max_iterations = 10:** The tree search performs up to ten refinement and debugging iterations per run.
- **$\epsilon = 0.4$ and debug_prob = 0.3:** The search algorithm probabilistically balances exploiting the current highest-scoring node against exploring alternative code candidate branches.
- **timeout = 3600s:** The per-node execution timeout was set to 3,600 seconds, which was never exceeded in any of our test runs.

The default LLM model specified in `config.toml` was overridden per run to evaluate performance across backends.

Data Collection & Tracked Measurements.

For every autonomous run, we logged comprehensive execution traces and output artifacts, recording:

- **Agent Satisfaction:** Whether the search tree successfully reached a node meeting the internal agent reviewer threshold (Score ≥ 80).
- **Quantitative Metrics:** Per-seed values for NDCG@10 and Recall@10 for all completed experiments, alongside summary statistics: mean, standard deviation, and coefficient of variation.
- **Code Complexity & Overhead:** Total lines of code (LOC) in the final candidate node and total wall-clock execution runtime.
- **Runtime Error Logs:** Every `stdout/stderr` trace, unhandled exception message, and failure mode encountered during the tree search.

Our forked code base, including configuration files and execution logs, is available at <https://github.com/BraelNamekong/AutoRecLab>.

2.2 Run Verification Process

We did not treat a run as successful simply because AutoRecLab marked it as satisfactory. After each run, we manually audited the generated output, the report, the execution logs, and the Python code (`runfile.py`). A run was only classified as achieving Methodological Success if it passed four sequential auditing checks.

Technical Execution Check (Execution Success).

First, we verified whether the run executed to completion without encountering fatal errors. A run achieved Execution Success only if the generated script executed cleanly and produced final output artifacts. Runs were flagged as execution failures if they encountered unhandled runtime exceptions, missing dependencies, missing dataset files, LensKit API errors, or system-level crashes, such as `ModuleNotFoundError`, `FileNotFoundError`, `AttributeError`, or `Windows BrokenProcessPool` exceptions.

Task and Dataset Fidelity Check.

Second, we checked whether the generated code faithfully implemented the task specified in our prompt:

- **Dataset Verification:** We confirmed that the script loaded the genuine MovieLens 100K u.data file, e.g., `data_path = os.path.join(..., 'u.data')`. We verified that the code did not replace the target dataset with synthetic or randomly generated data, such as creating mock user-item interaction arrays via `np.random.rand(...)`.
- **Seed-Sensitivity & Seed-Source Audit:** We verified that the script actively executed multiple random seeds inside a loop, for example `seeds = [1, 10, 3, 9, 5]`, rather than merely printing hardcoded seed numbers. Our audit explicitly categorized how the script applied those seeds, distinguishing between:
 - **Data-Split Sensitivity:** Passing random seeds to govern train/test dataset partitioning across iterations.
 - **Model/Training Seed Stochasticity:** Keeping the dataset split fixed and passing random seeds to control algorithm initialization or optimization in LensKit’s `ImplicitMF`.

Metric Traceability and Calculation Check. Third, we checked whether reported metrics originated from actual script computations rather than text hallucinated by the LLM report generator. We cross-referenced the AutoRecLab report, `stdout` logs, and `runfile.py` script.

Reported values had to be directly traceable to script outputs. For example, if a report claimed $\text{Mean NDCG@10} = 0.215$, we verified that the script calculated NDCG values per seed and averaged them. A methodologically sound script computes metrics explicitly (e.g., `def ndcg_at_k(...)` and `def recall_at_k(...)`). Hard-coded print statements like `print("Mean NDCG@10: 0.215")` were flagged as unverified. We also inspected scripts for silent calculation errors, such as misapplying `sklearn.metrics.ndcg_score` directly across raw score matrices instead of evaluating recommendations on a per-user basis.

Evaluation Protocol Check.

Fourth, we audited the evaluation protocol and test-set construction logic. We examined whether the script constructed test targets via global interaction splits, e.g., `train_test_split` on interaction rows, or user-level item holdouts.

For NDCG@10 and Recall@10 , we audited whether the evaluation logic correctly matched the chosen split structure. Specifically, we checked whether the code generated Top-10 recommendation lists for users, e.g., using `recommend_topk(..., k=10)` or ranking unobserved items, evaluated those lists against held-out test targets,

and calculated NDCG@10 and Recall@10 without data leakage or protocol mismatches.

Summary of Outcome Tiers.

Based on our validation process, every end-to-end run was classified into one of three outcome tiers:

- **Execution Success:** The run completed technically, generating code that executed to completion without runtime exceptions or missing dependency crashes.
- **AutoRecLab Satisfaction:** The run completed its search tree and produced a final node that met AutoRecLab’s internal reviewer threshold (Reviewer Score ≥ 80).
- **Methodological Success:** The run passed all four manual auditing checks—loading the genuine MovieLens 100K dataset, executing active multi-seed iterations, producing verifiable metric computations traceable to code, and applying a sound Top-N evaluation protocol without silent errors.

Only runs that passed all four checks achieved Methodological Success and were retained for downstream analysis.

2.3 Additional Verification Runs

In order to verify that AutoRecLab’s reported metrics stem from genuine computational execution rather than LLM text hallucination or hardcoded console outputs, we conducted three supplementary control runs. These runs serve as a dedicated verification benchmark (distinct from the 13 primary evaluation runs analyzed in Section 3) to empirically confirm that manually re-executing generated code yields 100% numerically identical results to the agent’s reported outputs. For all three supplementary verification runs, we selected gpt-5.4 as the LLM backend engine. We chose gpt-5.4 because it demonstrated the highest baseline reasoning capabilities and code synthesis stability among the evaluated backends during initial runs, minimizing technical infrastructure failures and establishing a reliable control environment to isolate execution fidelity.

Each member of our group conducted a separate AutoRecLab end-to-end run solely to verify that the final metrics stem from real computation. To illustrate this verification process, we examine a representative run conducted under the gpt-5.4 backend setup. Following tree search completion and agent satisfaction (Reviewer Score ≥ 80), we extracted the generated Python script (runfile.py) directly from the isolated Docker workspace.

Inspecting the runfile.py confirmed that the script executed real computational logic: it loaded MovieLens 100K (u.data), performed an 80/20 train/test split, trained LensKit’s ImplicitMF across ten random seeds, and evaluated Top-10 recommendations using Recall@10 and NDCG@10. Crucially, code inspection clarified the precise source of randomness being tested: the split used a deterministic, fixed seed (2024) for the 80/20 train/test division, while the ten random seeds varied model initialization and training stochasticity. To confirm that AutoRecLab’s final report was derived directly from this script’s execution, we manually re-executed runfile.py inside the exact same Docker container environment using the command: `uv run python runfile.py`.

We captured the complete, raw stdout terminal trace into a text artifact (manual_run_output.txt). Cross-referencing the per-seed

metric values in manual_run_output.txt against the summary report generated by AutoRecLab revealed 100% numerical exactness across all ten seeds. Table 1 summarizes the configuration parameters for this representative verification run, while Table 2 demonstrates that AutoRecLab’s reported Recall@10 and NDCG@10 values matched our manual rerun output line by line. The complete stdout comparison file for this run is archived in our repository as manual_verification_comparison_RUN5_GPT_5.4.txt. Setup parameters and manual rerun comparison logs for the remaining two supplementary verification runs conducted by team members are documented in the Appendix and archived in our GitHub repository <https://github.com/BraelNamekong/AutoRecLab.git>.

Table 1: Setup example from Member3_RUN5_AVR

Aspect	Setup
Run type	Additional Docker-based verification run
Manual command	<code>uv run python runfile.py + tee manual_run_output.txt</code>
Dataset	MovieLens100K u.data
Split	Fixed random 80/20 train/test split
Split seed	2024
Model	LensKit ImplicitMF
Hyperparameters	features=20, iterations=15, reg=0.1, weight=40, method=cg
Seeds	1, 7, 21, 42, 84, 126, 168, 210, 252, 294
Metrics	Recall@10 and NDCG@10
Purpose	Compare AutoRecLab report with manual runfile execution

Table 2: Per-seed comparison for the example manual verification run.

Seed	AR Recall	Manual Recall	AR NDCG	Manual NDCG	Match
1	0.1754	0.1754	0.2401	0.2401	Yes
7	0.1776	0.1776	0.2447	0.2447	Yes
21	0.1752	0.1752	0.2422	0.2422	Yes
42	0.1834	0.1834	0.2467	0.2467	Yes
84	0.1774	0.1774	0.2459	0.2459	Yes
126	0.1751	0.1751	0.2433	0.2433	Yes
168	0.1784	0.1784	0.2437	0.2437	Yes
210	0.1775	0.1775	0.2450	0.2450	Yes
252	0.1726	0.1726	0.2376	0.2376	Yes
294	0.1735	0.1735	0.2375	0.2375	Yes

Verification Findings & Scope Limitation

Across all three supplementary control checks, independent manual execution of runfile.py reproduced the reported metrics with zero numerical discrepancy. This proves execution fidelity, confirming that AutoRecLab’s outputs originate from authentic code execution rather than LLM text hallucinations. We emphasize, however, a critical distinction: while a manual rerun proves that a script ran and calculated the reported numbers, evaluating whether the script’s scientific logic is methodologically sound remains subject to the manual inspection protocol described in Section 2.2.

3 RESULTS & DISCUSSION

3.1 Overview of Results Across 13 Group Runs

To evaluate AutoRecLab, we conducted 13 initial end-to-end runs using the example prompt provided in the AutoRecLab README.md. The prompt instructs AutoRecLab to perform a seed-sensitivity analysis on LensKit’s ImplicitMF algorithm using the MovieLens 100K u.data dataset. We audited each run following the verification process described in Section 2.2; Table 3 summarizes the outcomes for all 13 primary group runs.

Comprehensive per-run specification tables, including full execution parameters, tree-search node lineage, and GitHub artifact links, are documented in Appendix.

Table 3: Summary of the 13 initial Group Runs.

Run ID	LLM Backend	OS / Environment	Reviewer Score	AutoRecLab Satisfactory (≥ 80)?
Member1_RUN1	gpt-5.4	Docker / Linux	86	Yes
Member1_RUN2	gpt-5.4-mini	Docker / Linux	80	Yes
Member1_RUN3	gpt-5-nano	Docker / Linux	58	No
Member1_RUN4	gpt-5.3-codex	Docker / Linux	N/A	No
Member2_RUN1	gpt-5.4-mini	Native Windows	N/A	No
Member2_RUN2	gpt-5.4-mini	Native Windows	82	Yes
Member2_RUN3	gpt-5.4	Native Windows	Accepted (Node 8)	Yes
Member2_RUN4	gpt-5-nano	Native Windows	Accepted (Node 7)	Yes
Member2_RUN6	gpt-5.3-codex	Native Windows	N/A	No
Member3_RUN1	gpt-5.4	Native Windows	83	Yes
Member3_RUN2	gpt-5.4-mini	Native Windows	85	Yes
Member3_RUN3	gpt-5.3-codex	Native Windows	N/A	No
Member3_RUN4	gpt-5-nano	Native Windows	90	Yes

Final Audit Status	Main Audit Observation
Methodologically Successful	Model seeds varied; split seed fixed (2024); passed all checks.
Methodologically Successful	Model seeds varied; split seed fixed (42); passed all checks.
Execution Failure	Failed tree search; syntax errors & missing <code>lenskit_implicit</code> .
Execution Failure	System API crash (NotFoundError: chat-completions endpoint incompatibility).
Execution Failure	Windows crash (BrokenProcessPool, AttributeError, fallback IndexError).
Execution Success Only	Silent metric failure: misapplied <code>sklearn.ndcg_score</code> to raw score vectors.
Methodologically Successful	Model seeds varied; split seed fixed (2024); resolved Windows parallel errors.
Execution Success Only	Protocol deviation: fell back to custom SGD matrix factorization instead of ImplicitMF.
Execution Failure	System API crash (NotFoundError: chat-completions endpoint incompatibility).
Methodologically Successful	Model seeds varied; split seed fixed (2025); passed all checks.
Methodologically Successful	Model seeds varied; split seed fixed (42); passed all checks.
Execution Failure	System API crash (NotFoundError: chat-completions endpoint incompatibility).
Execution Success Only	Protocol deviation: algorithm/fallback uncertainty; failed requested task fidelity.

Analyzing our 13 primary group runs through the three auditing tiers reveals a clear gap between technical execution, agent self-assessment, and true methodological validity:

- **Execution Success Tier:** Out of 13 initial runs, 8 runs achieved technical execution success by running to completion and generating output artifacts. The remaining 5 runs failed during execution due to fatal API errors, missing library imports, or operating system process crashes.
- **AutoRecLab Satisfaction Tier:** AutoRecLab’s internal reviewer assigned a satisfactory score (≥ 80 or accepted node) to 8 runs, marking them as successful implementations within its tree search.
- **Methodological Success Tier:** Manual inspection following Section 2.2 revealed that exactly 5 out of 13 runs achieved true Methodological Success. The remaining 3 completed runs that AutoRecLab marked as satisfactory (Member2_RUN2,

Member2_RUN4, and Member3_RUN4) contained critical scientific flaws, such as misapplying `sklearn.ndcg_score` across raw score vectors or abandoning LensKit’s ImplicitMF for custom SGD fallbacks, that went completely undetected by the agent’s internal reviewer.

Performance by Backend Engine.

Run stability and script quality varied significantly across the evaluated LLM backends:

- **gpt-5.4:** Proved to be the most reliable backend. All 3 initial runs (Member1_RUN1, Member2_RUN3, and Member3_RUN1) achieved Execution Success, received satisfactory reviewer scores, and passed all manual audit checks to reach Methodological Success.
- **gpt-5.4-mini:** Showed strong capabilities but higher variance. Out of 4 runs, 2 achieved Methodological Success (Member1_RUN2 and Member3_RUN2). However, 1 run suffered an execution crash under Windows multiprocessing (Member2_RUN1), and another produced code with a silent metric evaluation error (Member2_RUN2).
- **gpt-5-nano:** Struggled consistently with library integration across all 3 runs (Member1_RUN3, Member2_RUN4, and Member3_RUN4), achieving 0 Methodologically Successful runs. When LensKit imports failed, scripts generated by gpt-5-nano systematically abandoned the requested ImplicitMF algorithm and fell back to custom SGD matrix factorization scripts, deviating from the prompt protocol.
- **gpt-5.3-codex:** Suffered a 100% failure rate across all 3 attempts (Member1_RUN4, Member2_RUN6, and Member3_RUN3). All runs failed at the initial API request stage due to endpoint incompatibility (NotFoundError) before generating any code or search nodes.

These findings highlight that an agent’s internal reviewer score cannot be taken at face value. The specific failure modes fall into two categories: hard technical execution crashes, analyzed in Section 3.2, and silent methodological errors, analyzed in Section 3.3.

3.2 Technical Execution Errors and Runtime Crashes

Out of our 13 initial runs, 5 runs experienced fatal technical execution failures and were unable to run to completion or produce evaluation metrics. These execution crashes occurred before reaching a satisfactory tree search node and fall into three distinct technical error categories: API endpoint incompatibilities, OS-specific multiprocessing crashes, and library dependency failures.

1. API Endpoint Incompatibility Errors.

All three attempts using the *gpt-5.3-codex* backend (Member1_RUN4, Member2_RUN6, and Member3_RUN3) failed immediately upon launching. The underlying framework threw an `OpenAI NotFoundError`, stating that the specified model was not supported by the chat-completions endpoint. Because these failures occurred during the initial API connection phase (before any code generation or tree search iterations could begin) 0 seeds were executed, and no workspace artifacts were generated.

2. Operating System and Multiprocessing Locks (Native Windows Environment).

Executing AutoRecLab in a native Windows environment introduced platform-specific process concurrency issues. In *Member2_RUN1* (*gpt-5.4-mini* on Windows), parallel execution tasks across multiple tree search nodes repeatedly crashed with *BrokenProcessPool* exceptions alongside *AttributeError* and *TypeError* exceptions. When AutoRecLab attempted an automated fallback to execute the best evaluated node, the script terminated with *IndexError: list index out of range*, failing to produce numeric metrics. This crash demonstrated how native Windows process spawning (*spawn* vs. Linux *fork*) disrupts LensKit’s parallel seed evaluation unless worker scripts explicitly handle Windows multiprocessing guards.

3. Library Dependency and Code Syntax Failures (*gpt-5-nano*).

In *Member1_RUN3* (*gpt-5-nano* on Docker/Linux), AutoRecLab attempted 5 tree search iterations but failed to produce a single runnable node. The Python scripts synthesized by *gpt-5-nano* contained recurring *SyntaxError* bugs, runtime exceptions declaring “*ImplicitMF is not available*”, and *ModuleNotFoundError: no module named lenskit_implicit*. Because the model failed to format the LensKit import statements correctly, the tree search stalled at a peak reviewer score of 58, exiting without generating valid metrics.

These 5 execution failures highlight that OS selection, endpoint compatibility, and backend LLM choice heavily dictate whether AutoRecLab can successfully execute. However, as analyzed in Section 3.3, technical completion alone does not guarantee scientific accuracy.

3.3 Methodological Flaws and Silent Calculation Errors

While 8 out of 13 initial runs achieved technical execution success, manual verification following Section 2.2 revealed that 3 of these completed runs failed scientific auditing despite receiving high internal reviewer scores from AutoRecLab (≥ 80).

These runs represent the most critical findings of our evaluation: AutoRecLab can generate runnable code and assign high self-evaluation scores to scripts that contain severe methodological errors.

Table 4: Summary of Methodologically Flawed Completed Runs.

Run ID	LLM Backend	Reviewer Score	Primary Failure Category
Member2_RUN2	gpt-5.4-mini	82	Silent Metric Failure
Member2_RUN4	gpt-5-nano	Accepted (Node 7)	Protocol Deviation
Member3_RUN4	gpt-5-nano	90	Protocol Uncertainty

Specific Scientific Impact

Applied *sklearn.metrics.ndcg_score* on score vectors; truncated seeds; reported flawed NDCG@10 of 0.4080 ± 0.0046 (Recall@10: 0.1859 ± 0.0042).

LensKit import failed; fell back to custom SGD-MF ($n=20$, $lr=0.005$); reported NDCG@10 of 0.0234 ± 0.0008 .

Fallback recommender usage; uncertain algorithm fidelity; reported NDCG@10 of 0.2155 ± 0.0036 .

1. Silent Metric Calculation Errors.

In *Member2_RUN2* (*gpt-5.4-mini* on Windows), AutoRecLab produced a script that executed without throwing runtime exceptions

and earned an internal reviewer score of 82, despite only generating outputs for 5 out of the 10 expected model seeds. However, manual code inspection revealed that the script misapplied *sklearn.metrics.ndcg_score* directly to predicted score vectors rather than computing Top-N recommendation NDCG over ranked unseen items.

This error resulted in a reported NDCG@10 of 0.4080 ± 0.0046 , which is approximately twice the expected range for LensKit’s *ImplicitMF* on MovieLens 100K. Because the code threw no runtime exceptions and generated realistic-looking numeric outputs, AutoRecLab’s internal reviewer accepted the output as a valid solution without flagging the conceptual error.

2. Unprompted Algorithm Fallbacks and Protocol Drift.

In both *Member2_RUN4* and *Member3_RUN4* (*gpt-5-nano* on Windows), AutoRecLab struggled to import LensKit’s *ImplicitMF* module. Rather than reporting an execution failure or resolving the library configuration, AutoRecLab autonomously modified the experiment protocol:

- In *Member2_RUN4*, the synthesized script silently substituted LensKit’s *ImplicitMF* with a custom SGD-based matrix factorization implementation using arbitrary hyperparameters ($n_factors=20$, $lr=0.005$, $reg=0.001$, $epochs=8$, $\alpha=0.5$). This yielded a near-zero NDCG@10 of 0.0234 ± 0.0008 and truncated the evaluation to only 5 of the 10 requested seeds.
- In *Member3_RUN4*, the script introduced fallback logic that created uncertainty around which algorithm was actually evaluated, abandoning requested task fidelity.

Despite completely failing to evaluate the requested LensKit *ImplicitMF* algorithm, AutoRecLab accepted these runs into its tree search and assigned *Member3_RUN4* an internal reviewer score of 90.

These three runs demonstrate why AutoRecLab’s internal evaluation score cannot serve as a stand-in for human scientific review. AutoRecLab’s internal reviewer evaluates whether outputs look structural, plausible, and complete, but it fails to detect logic-level metric errors or subtle protocol deviations introduced during script synthesis.

3.4 Recommender Findings, Seed Sensitivity, and Qualitative Comparison Against Preprint Claims

Following our multi-tier audit, exactly 5 out of 13 initial runs achieved Methodological Success (*Member1_RUN1*, *Member1_RUN2*, *Member2_RUN3*, *Member3_RUN1*, and *Member3_RUN2*). In these 5 runs, AutoRecLab correctly evaluated LensKit’s *ImplicitMF* algorithm on the MovieLens 100K dataset, held the train/test split protocol fixed, and systematically varied model training seeds to conduct the requested seed-sensitivity analysis.

Table 5 details the quantitative recommender performance metrics, seed configurations, and protocol details for all methodologically successful runs.

Table 5: Seed Sensitivity and Metric Summary for Methodologically Successful Runs.

Run ID	LLM Backend	Source of Randomness	Split / Metric protocol
Member1_RUN1	gpt-5.4	Model / Training Seed	Fixed Split (2024) / Top-10
Member1_RUN2	gpt-5.4-mini	Model / Training Seed	Fixed Split (42) / Top-10
Member2_RUN3	gpt-5.4	Model / Training Seed	Fixed Split (2024) / Top-10
Member3_RUN1	gpt-5.4	Model / Training Seed	Fixed Split (2025) / Top-10
Member3_RUN2	gpt-5.4-mini	Model / Training Seed	Fixed Split (42) / Top-10

Seeds Available / Expected	Ex- NDCG@10 (Mean $\pm$ SD)	Recall@10 (Mean $\pm$ SD)
10 / 10	0.1987 $\pm$ 0.0038	0.1892 $\pm$ 0.0030
10 / 10	0.2312 $\pm$ 0.0028	0.1703 $\pm$ 0.0014
10 / 10	0.2115 $\pm$ 0.0052	0.3461 $\pm$ 0.0063
10 / 10	0.1910 $\pm$ 0.0031	0.2984 $\pm$ 0.0044
10 / 10	0.2353 $\pm$ 0.0021	0.1719 $\pm$ 0.0025

Across all 5 methodologically successful runs, varying model training seeds produced low standard deviations, showing that model-level seed variance remains tight when data splits are kept constant.

Sensitivity to Data Split Protocols.

While model seed variance was low within any single run, absolute metric values differed noticeably depending on the specific split seed and exact data partitioning choices made by AutoRecLab:

- Runs using split seed 42 (*Member1_RUN2* and *Member3_RUN2*) showed consistent results across different LLM backends, yielding NDCG@10 scores of ~ 0.231 to ~ 0.235 and Recall@10 scores of ~ 0.170 to ~ 0.171 .
- However, runs using split seed 2024 (*Member1_RUN1* and *Member2_RUN3*) exhibited performance divergence, with *Member1_RUN1* producing an NDCG@10 of 0.1987 ± 0.0038 and *Member2_RUN3* producing an NDCG@10 of 0.2115 ± 0.0052 .

This variance underscores that data partitioning stochasticity and underlying train/test stratification logic written during script synthesis exert a substantial impact on evaluation metrics. Because AutoRecLab selected fixed split seeds autonomously per run rather than standardizing split seeds globally across all group experiments, cross-run comparison requires careful accounting for these underlying partition differences.

Qualitative Comparison Against the AutoRecLab Preprint and README Vision.

Because our evaluation was limited to 13 initial group runs on a single dataset (MovieLens 100K), our findings cannot serve as a sweeping judgment of AutoRecLab in all possible contexts. However, looking at what our experiments did and did not prove provides a clear, grounded picture:

- **What Our Experiments Confirmed:** AutoRecLab demonstrates exceptional competence in rapid experiment scaffolding, dependency resolution, and automated code generation. When the tree-search mechanism successfully navigates away from structural blockers, the agent is capable of executing complex multi-seed evaluations and producing valid baseline recommender pipelines without manual intervention.

- **Where Results Diverged from Preprint Claims:** The preprint and README emphasize robust autonomous self-governance, reliable internal self-e group evaluation, this claim was challenged. AutoRecLab’s internal reviewer frequently awarded high scores (≥ 80) to scripts containing critical conceptual flaws, such as vector-level sklearn.ndcg_score misapplications (*Member2_RUN2*) or unprompted algorithmic substitutions (*Member2_RUN4* and *Member3_RUN4*). This indicates that the agent’s internal critique mechanism is overly lenient and lacks deep scientific validation capabilities.
- **Attribution of Operational Limitations:** Certain execution breakdowns observed during our evaluation (such as native Windows multiprocessing crashes “*BrokenProcessPool*”) were partially tied to host environment idiosyncrasies rather than core algorithmic flaws in AutoRecLab. However, the agent’s tendency to execute silent protocol drifts (falling back to custom SGD matrix factorization when LensKit imports failed) rather than raising explicit configuration errors highlights a structural limitation in AutoRecLab reliability.

Within the scope of our pilot-scale evaluation, AutoRecLab functions optimally as an accelerated code-drafting and experimental prototyping assistant rather than an independent, self-sustaining research scientist. Bridging the gap between the fully autonomous “research lab” vision and empirical reliability requires strict, mandatory human-in-the-loop auditing to intercept silent metric errors and verify experimental protocol fidelity.

3.5 Overall Discussion: Assessing AutoRecLab’s Capabilities

Our group evaluation provides a comprehensive look at how AutoRecLab performs under a standardized research prompt. Synthesizing our findings allows us to answer our core research questions regarding AutoRecLab’s operational viability and scientific reliability:

End-to-End Execution and Model Dependence: AutoRecLab demonstrates a genuine capability to plan, implement, and execute recommender-systems experiments autonomously. Across our 13 group runs, 8 reached a satisfactory node according to AutoRecLab’s internal criteria, but strictly 5 runs achieved Methodological Success after our rigorous manual auditing. This operational outcome is strongly dictated by the choice of LLM backend and execution environment. Stronger models like gpt-5.4 achieved complete reliability, while smaller or incompatible configurations encountered structural roadblocks.

Comparison with Published Paper and README Promises: Our group findings align with the core qualitative conclusions of the original README and preprint. While our focused README prompt ran faster (13 to 30 minutes) due to a smaller experimental scope than the preprint’s broader grid, both investigations arrived at the same critical takeaway: random seeds measurably influence recommender systems performance, reinforcing the community requirement for multi-seed reporting. Furthermore, our manual

verification runs confirmed that successful outputs produce reproducible, code-driven metrics rather than text-based hallucinations.

Why Human-in-the-Loop Auditing Remains Essential: Despite its strengths, AutoRecLab exhibits a critical blind spot regarding silent methodological errors. While it successfully debugs explicit syntax or import exceptions, it can still assign high reviewer scores to scripts containing flawed evaluation logic, such as vector-level metric misapplications. Consequently, our evaluation concludes that while AutoRecLab functions as a highly effective exploratory assistant, it is not yet reliable enough for unsupervised, production-level research without rigorous human-in-the-loop methodological governance.

Ultimately, our findings reinforce the overarching call to action put forward by the preprint: the recommender systems community needs to actively embrace and invest in autonomous research agents, mirroring the rapid progress seen in other AI disciplines. Both the original preprint and our group evaluation serve as an encouraging starting point and a clear proof of concept, demonstrating that autonomous experimentation in recommender systems is genuinely feasible. Rather than dismissing these systems due to early limitations, the RecSys community should look more deeply into this emerging paradigm, actively contributing to the development, refinement, and safe governance of autonomous research labs.

4 LIMITATIONS

While our evaluation provides valuable empirical insights into AutoRecLab 0.1, several methodological and structural limitations should be taken into account when interpreting our findings:

Focused Experimental Scope.

We restricted our group evaluation to the README-suggested seed-sensitivity experiment using MovieLens 100K and LensKit’s ImplicitMF algorithm. In contrast, the original preprint evaluated a broader experimental grid across multiple datasets and algorithms. Consequently, our 13-run study represents a focused pilot reproduction rather than a full replication or macro-benchmark of the entire platform.

Intentional Omission of Hyperparameter Optimization.

We deliberately omitted hyperparameter tuning across our group runs. This decision was necessary to isolate the effects of random seeds and LLM agent code-generation behavior. Introducing hyperparameter searches would have added another layer of variance, making it difficult to determine whether performance shifts stemmed from model training seeds, parameter choices, or AutoRecLab’s synthesized execution logic.

Environment and Protocol Variability.

Our group experiments exposed platform-dependent constraints, particularly when running AutoRecLab natively on Windows hosts where multiprocessing concurrency crashes (BrokenProcessPool) interrupted execution. Additionally, because split seeds were selected autonomously by the agent per run rather than standardized globally across team members, cross-run comparison required careful accounting for underlying partition differences.

Generalizability and Future Work.

Our findings are specific to the tested seed-sensitivity task, the tested LLM backends, and AutoRecLab 0.1. We cannot extrapolate these reliability findings to other recommender algorithms, complex multi-dataset tasks, or future versions of the agent. Future research should evaluate autonomous research agents on standardized, multi-dataset benchmarks with fixed evaluation protocols, explicit environment containerization, and formal human-in-the-loop verification frameworks.

5 ACKNOWLEDGEMENT OF AI

We thank Prof. Joeran Beel for supervising this work, assigning the research topic and organizing access to the OpenAI API for the course. The initial draft of this report was generated with Claude based on our experimental logs and presentation materials. All subsequent editing, restructuring, fact checking, and validation were performed by our group.

REFERENCES

- [1] Joeran Beel and Moritz Baumgart. 2026. Intelligent Systems Seminar. *Universitt Siegen* (2026).
- [2] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, Philipp Meister, and Sinan Pourazari. 2026. AutoRecSys or AutoRecLab? A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs, and a First Proof-of-Concept. *ACM Transactions on Recommender Systems*, preprint. Preprint.
- [3] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, et al. 2025. Towards an AI Co-Scientist. Google DeepMind technical report.
- [4] Peter Jansen et al. 2025. CodeScientist: End-to-End Semi-Automated Scientific Discovery with Code-based Experimentation. *arXiv preprint*.
- [5] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.
- [6] Samuel Schmidgall et al. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. *arXiv preprint*.

APPENDIX

The full run matrices are provided below.

Table 6: Run matrix for Member1_RUN1.

Field	Content
Run-ID	Member1_RUN1
Experiment phase	initial
Model and environment	gpt-5.4, Docker/Linux
AutoRecLab status	Satisfactory node reached, reviewer score 86
Technical status	Completed, numeric metrics produced
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	node 3, selected after tree-search iteration 4/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/AutoRecLab_Lopez_Runs_Main/Run1-Lopez%20(gpt-5.4)
Own verification	inspection

Table 7: Run matrix for Member1_RUN2.

Field	Content
Run-ID	Member1_RUN2
Experiment phase	initial
Model and environment	gpt-5.4-mini, Docker/Linux
AutoRecLab status	Satisfactory node reached, reviewer score 80
Technical status	Completed, numeric metrics produced
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	node 3, selected after tree-search iteration 1/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/AutoRecLab_Lopez_Runs_Main/Run2-Lopez%20(gpt-5.4-mini)
Own verification	inspection

Table 8: Run matrix for Member1_RUN3.

Field	Content
Run-ID	Member1_RUN3
Experiment phase	initial
Model and environment	gpt-5-nano, Docker/Linux
AutoRecLab status	No satisfactory node reached; best reviewer score 58
Technical status	Not completed. Repeated failures (SyntaxError, RuntimeError "ImplicitMF is not available", ModuleNotFoundError: no module named lenskit_implicit). No node produced metric values. The archived log ends during tree-search iteration 5/10.
Scientific status	Invalid / Excluded (No Satisfactory Node / Fallback Attempt)
Expected / received seeds	unclear / 0
Exclusion reason	No satisfactory node; LensKit ImplicitMF unavailable, generated code fell back to a custom SGD matrix factorization instead of the requested algorithm; no valid metrics produced
Used node	None
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/AutoRecLab_Lopez_Runs_Main/Run3-Lopez%20(gpt-5-nano)
Own verification	inspection

Table 9: Run matrix for Member1_RUN4.

Field	Content
Run-ID	Member1_RUN4
Experiment phase	initial
Model and environment	gpt-5.3-codex, Docker/Linux
AutoRecLab status	No node
Technical status	Failed with OpenAI NotFoundError: model not supported in chat-completions endpoint
Scientific status	Invalid / Excluded (System / API Failure)
Expected / received seeds	10 / 0
Exclusion reason	endpoint incompatibility
Used node	None
Result source	No artifacts archived; the run failed at the first API call, before any node or code was generated
Own verification	inspection

Table 10: Run matrix for Member1_RUN5_AVR.

Field	Content
Run-ID	Member1_RUN5_AVR
Experiment phase	Additional verification run with a manual execution of generated runfile.py
Model and environment	gpt-5.4, Docker/Linux
AutoRecLab status	Satisfactory node reached, reviewer score 92
Technical status	Completed, numeric metrics produced
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	node 3, selected after tree-search iteration 3/10
Result source	https: //github.com/BraelNamekong/AutoRecLab/tree/main/out/AutoRecLab_Lopez_Runs_Main/RUN_5_Additional_Verification_Run https://github.com/BraelNamekong/AutoRecLab/tree/main/workspace/RUN_5_Additional_Verification_Run_Nicolas_Lopez
Own verification	inspection + Manual execution of generated code + Comparison

Table 11: Run matrix for Member2_RUN1.

Field	Content
Run-ID	Member2_RUN1
Experiment phase	initial
Model and environment	gpt-5.4-mini, Windows
AutoRecLab status	No satisfactory node reached
Technical status	Execution crashed with BrokenProcessPool and AttributeError/TypeError across multiple nodes; the final fallback to the best node crashed with IndexError: list index out of range, so no metrics were produced.
Scientific status	Invalid
Expected / received seeds	10 / 0
Exclusion reason	Tree search crashed before any usable node was produced
Used node	None
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Brael_Runs_Main/Run1
Own verification	inspection

Table 12: Run matrix for Member2_RUN2.

Field	Content
Run-ID	Member2_RUN2
Experiment phase	retry of RUN1 (after a fixable environment failure)
Model and environment	gpt-5.4-mini, Windows
AutoRecLab status	Satisfactory node found, reviewer score 82
Technical status	Completed, numeric metrics produced
Scientific status	Invalid - Excluded: wrong metric
Expected / received seeds	10 / 5
Exclusion reason	The generated code used sklearn.ndcg_score on score vectors instead of computing Top-N recommendation NDCG from ranked unseen items. The resulting NDCG@10 of 0.4080 is about twice the plausible range for ImplicitMF on MovieLens 100K. This is the silent metric failure discussed in the report; the run passed execution and received a high reviewer score but measured the wrong concept.
Used node	Node found at tree-search iteration 4/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Brael_Runs_Main/Run2
Own verification	inspection

Table 13: Run matrix for Member2_RUN3.

Field	Content
Run-ID	Member2_RUN3
Experiment phase	initial
Model and environment	gpt-5.4, Windows
AutoRecLab status	Node 8 executed successfully (1/12 nodes succeeded)
Technical status	Initial nodes failed due to matplotlib missing and LensKit API/multiprocessing issues. Node 8 resolved all errors using Recommender.adapt() and serial evaluation.
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	Node 8
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Brael_Runs_Main/Run3
Own verification	inspection

Table 14: Run matrix for Member2_RUN4.

Field	Content
Run-ID	Member2_RUN4
Experiment phase	initial
Model and environment	gpt-5-nano, Windows
AutoRecLab status	Completed via fallback (LensKit ImplicitMF unavailable; multiple earlier nodes failed)
Technical status	Completed using a custom SGD-based matrix factorisation fallback instead of LensKit ImplicitMF, producing traceable numeric metrics
Scientific status	Methodologically Invalid due to protocol deviation (SGD fallback instead of LensKit ImplicitMF)
Expected / received seeds	10 / 5
Exclusion reason	Protocol deviation: LensKit repeatedly failed to import, so the generated code fell back to a custom SGD matrix factorisation with its own hyperparameters (n_factors=20, lr=0.005, reg=0.001, epochs=8, alpha=0.5). The requested LensKit ImplicitMF was therefore not evaluated.
Used node	Node 7 (SGD-fallback node - after several failed LensKit-based attempts)
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Brael_Runs_Main/Run4 https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Brael_Runs_Main/working/results
Own verification	inspection

Table 15: Run matrix for Member2_RUN6.

Field	Content
Run-ID	Member2_RUN6
Experiment phase	initial
Model and environment	gpt-5.3-codex, Windows
AutoRecLab status	No node
Technical status	Failed with OpenAI NotFoundError: model not supported in chat-completions endpoint
Scientific status	Invalid / Excluded (System / API Failure)
Expected / received seeds	10 / 0
Exclusion reason	endpoint incompatibility
Used node	None
Result source	No artifacts archived; the run failed at the first API call, before any node or code was generated
Own verification	inspection

Table 16: Run matrix for Member2_RUN5_AVR.

Field	Content
Run-ID	Member2_RUN5_AVR
Experiment phase	Additional verification run with a manual execution of generated runfile.py
Model and environment	gpt-5.4, Docker/Linux
AutoRecLab status	Satisfactory node found at tree-search iteration 7/10
Technical status	Completed successfully; all 10 seeds executed and produced numeric metrics
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	Node accepted at tree-search iteration 7/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Brael_Runs_Main/Run_5_Additional_Verification_Run https://github.com/BraelNamekong/AutoRecLab/tree/main/workspace/RUN_5_Additional_Verification_Run_Brael
Own verification	inspection + Manual re-execution of runfile.py + Comparison against the accepted node

Table 17: Run matrix for Member3_RUN1.

Field	Content
Run-ID	Member3_RUN1
Experiment phase	initial
Model and environment	gpt-5.4, Windows
AutoRecLab status	Satisfactory node reached, reviewer score 83
Technical status	Completed, numeric metrics produced
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	node 3, selected after tree-search iteration 3/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Amine_Bouhnini_Runs_Main/RUN1
Own verification	inspection

Table 18: Run matrix for Member3_RUN2.

Field	Content
Run-ID	Member3_RUN2
Experiment phase	initial
Model and environment	gpt-5.4-mini, Windows
AutoRecLab status	Satisfactory node reached, reviewer score 85
Technical status	Completed, numeric metrics produced
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	node 3, selected after tree-search iteration 6/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Amine_Bouhnini_Runs_Main/RUN2
Own verification	inspection

Table 19: Run matrix for Member3_RUN3.

Field	Content
Run-ID	Member3_RUN3
Experiment phase	initial
Model and environment	gpt-5.3-codex, Windows
AutoRecLab status	No node
Technical status	Failed with OpenAI NotFoundError: model not supported in chat-completions endpoint
Scientific status	invalid / Excluded (System / API Failure)
Expected / received seeds	10 / 0
Exclusion reason	endpoint incompatibility
Used node	None
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Amine_Bouhnini_Runs_Main/RUN3
Own verification	inspection

Table 20: Run matrix for Member3_RUN4.

Field	Content
Run-ID	Member3_RUN4
Experiment phase	initial
Model and environment	gpt-5-nano, Windows
AutoRecLab status	Satisfactory node reached, reviewer score 90
Technical status	Completed, result files produced
Scientific status	Invalid
Expected / received seeds	5 / 5
Exclusion reason	Unclear algorithm implementation, Run may not consistently use LensKit ImplicitMF because fallback recommender usage is possible
Used node	node 3, selected after tree-search iteration 5/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Amine_Bouhnini_Runs_Main/RUN4
Own verification	inspection

Table 21: Run matrix for Member3_AVR.

Field	Content
Run-ID	Member3_RUN5_AVR
Experiment phase	Additional verification run with a manual execution of generated runfile.py
Model and environment	gpt-5.4, Docker/Linux
AutoRecLab status	Satisfactory node reached, reviewer score 83
Technical status	Completed, numeric metrics produced
Scientific status	Methodologically Successful
Expected / received seeds	10 / 10
Exclusion reason	None
Used node	node 3, selected after tree-search iteration 1/10
Result source	https://github.com/BraelNamekong/AutoRecLab/tree/main/out/Autoreclab_Amine_Bouhnini_Runs_Main/RUN_5_Additional_Verification_Run
Own verification	https://github.com/BraelNamekong/AutoRecLab/tree/main/workspace/RUN_5_Additional_Verification_Run_Amine_Bouhnini inspection + Manual execution of generated code + Comparison